Protein Function Prediction Using Pretrained Language Model Embeddings and Machine Learning

Quinn Ramsay
High Point University
Department of Computer Science
High Point, North Carolina, USA
qramsay@highpoint.edu

Abstract

Proteins are the main driving force in cellular processes, yet when it comes to understanding protein sequences, less than 1% of all sequences have been experimentally verified. Experimental characterization is accurate, but a very slow and expensive process for identifying their function. With the advancements in machine learning technologies, scientific studies have turned to data scientists to try to create models that can accurately predict a protein's function faster than any experimental study. This project addresses automated protein function prediction by building a machine learning pipeline that predicts Gene Ontology (GO) term annotations from raw amino acid sequences.

The GO system organizes functional labels into three sub-ontologies: Molecular Function (MF), Biological Process (BP), and Cellular Component (CC). Protein sequences are embedded using ESM-2 and ProtT5 pretrained language models, producing three datasets: ESM-2 alone, ProtT5 alone, and a concatenated combination. Each dataset is evaluated using an XGBoost classifier and a Multilayer Perceptron. An MLP+XGBoost ensemble and a cross-embedding MLP stack were also tested. The best configuration, the MLP stack, achieved a CAFA score of 0.5063, surpassing the project's target benchmark of 0.472.

1 Introduction

Proteins form most of the world that are essential within our cells, yet there is still a sizable gap between the number of known protein sequences and our understanding of their function. While there have been lots of advancements in technology, which have rapidly improved the identification of new protein sequences, experimental pathways have remained slow and expensive. This has created a bottleneck in the biological research community, with such a vital part of science unknown; we are leaving millions of protein sequences with unknown or partially annotated functions, limiting our ability to understand cellular processes and disease mechanisms fully.

The motivation for this project stems from the opportunity to address a fundamental challenge in biological research and have a direct impact on the studies being done. While protein function prediction has been extensively studied and has appeared in previous Kaggle competitions, there remains substantial room for improvement.

The objective of this project is to create a model that can predict Gene Ontology (GO) functional terms based directly on the amino acid sequence of proteins. The model will take an input sequence consisting of the 20 standard letters of amino acids and produce an output consisting of predicted GO terms that describe the molecular function of the protein. The predicted function will also have a confidence level.

2 Related Work

Several research groups have worked on predicting protein function from amino acid sequences, using different methods and datasets.

Two survey papers give a broad view of the field. Kulmanov and Hoehndorf [1] cover the history of protein function prediction and explain how the field has moved from traditional methods to deep learning. Boadu et al. [7] focus specifically on deep learning methods and describe the most recent advances in the area. Earlier community-wide evaluations by Jiang et al. [11] and Friedberg and Radivojac [9] document the progression of the CAFA challenges and how prediction accuracy has improved over successive rounds.

Some recent papers use deep learning models that combine sequence data with protein structure information. Wang et al. [3] created DPFunc, a model that uses deep learning with domain-guided structure information to predict protein function. Their model reports strong results, but it depends heavily on AlphaFold2 predictions for the structure inputs. Meng and Wang [4] developed TAWFN, a framework that fuses sequence and structure data to predict function. TAWFN shows good results, but the authors did not release their code, which makes it hard to compare against directly.

Another research direction uses gradient boosting instead of neural networks. Iosipoi and Vakhrushev [13] created the Py-Boost library and its underlying algorithm, SketchBoost. This method was built to train gradient boosted decision trees on problems with thousands of output targets, which is normally very slow with standard tools like XGBoost [14]. Py-Boost was used by the second place team in the CAFA-5 challenge, which shows that it works well for protein function prediction. It is different from the XGBoost approach

used in this project because it trains one model for all targets at once instead of one model per target.

A few papers use mutational scanning data to train machine learning models. Sandhu et al. [2] study what factors affect machine learning performance on protein fitness prediction. Their architecture is interesting, but they focus on scalar fitness values instead of GO terms. Villegas-Morcillo et al. [5] use deep neural networks to learn sequence-function relationships from deep mutational scanning data. Their work shows that neural networks can learn nonlinear patterns in protein sequences, but they train on DMS data instead of GO annotations. Yan et al. [6] present PPVED, a plant-specific tool that predicts the effect of single amino acid substitutions. PPVED performs better than tools trained on human data for this specific task, but it only predicts whether a mutation is harmful or neutral, not full GO terms.

3 Dataset and Features

The dataset comes from the CAFA-6 Protein Function Prediction Kaggle competition. Gene Ontology (GO) is a structured system that describes the biological roles of genes and proteins. It organizes functions into three groups: Molecular Function (MF), Biological Process (BP), and Cellular Component (CC).

The training data includes two main files. The first file, `train_sequences.fasta`, contains 82,404 protein sequences written as amino acid strings. The second file, `train_terms.tsv`, has 537,027 rows that map each protein EntryID to a GO term and its GO group. Altogether, the dataset includes 26,125 unique GO terms.

Table 1 shows a sample of rows from `train_terms.tsv`, and Figure 1 shows the format of `train_sequences.fasta`.

Table 1. Sample rows from `train_terms.tsv`. Each row represents one GO term annotation for one protein. The aspect column uses single-letter codes where C stands for Cellular Component, F stands for Molecular Function, and P stands for Biological Process.

EntryID	term	aspect
Q5W0B1	GO:0000785	C
Q5W0B1	GO:0004842	F
Q5W0B1	GO:0051865	P
Q5W0B1	GO:0006275	P
Q5W0B1	GO:0006513	P

Two preprocessing steps were done before training. First, 200 sequences with nonstandard amino acids were removed. Second, GO terms with fewer than 50 examples were removed because there was not enough data for the model to learn from. This reduced the label space from 26,125 terms to 1,585 usable terms. These include 978 BP terms, 278 MF terms, and 329 CC terms.

Figure 1. Sample entries from `train_sequences.fasta`. Each protein entry begins with a header line starting with >, followed by the amino acid sequence. The EntryID used to match with `train_terms.tsv` is the middle field of the header.

```
>sp|Q5W0B1|RN169_HUMAN
MAEVAPAQWLLWGPQRELLKMFGAGCQSSTQPGKS
LLLKNTLSSSAPSGEKPDAEVLVGRCGLCQQPVPV
SRVAVHMASCQRGQWEQAQQQHLEQTKKGTKSIE
...

>sp|P12345|GOT2_HUMAN
MALLHSARVLSGVASAFHPGLAAAASARASSWW
THVEMGPPDPILGVTEAFKRDTNSKKMNLGVGAY
...
```

About 6,000 proteins only had rare labels, so they lost all annotations after filtering. Those proteins were dropped from the label matrix. The final dataset contains 76,458 proteins with at least one valid annotation.

4 Methodology

This project uses a pipeline with three main stages: embedding generation, classification, and ensemble combination. All methods used are existing algorithms and libraries. The pretrained language models (ESM-2 and ProtT5) were used as provided by their respective authors through Hugging Face Transformers [20]. The classification models (XGBoost and MLP) were implemented using the XGBoost and PyTorch [19] libraries. The contribution of this work is the design and evaluation of the full pipeline, including the systematic comparison of embedding types and classifiers. All code and configuration files for this project are available on GitHub [21].

4.1 Embedding Generation

Protein sequences cannot be fed directly into a machine learning model as text. They must first be converted into fixed-length numeric vectors called embeddings. This project uses two pretrained protein language models to generate embeddings.

The first model is ESM-2 (facebook/esm2_t33_650M_UR50D). It is a 650-million parameter transformer model developed by Meta and trained on hundreds of millions of protein sequences from the UniRef50 database [5]. Given a protein sequence, ESM-2 produces a hidden state vector at each position in the sequence. To obtain a single fixed-length representation for the entire protein, mean pooling is applied across all sequence positions, ignoring padding tokens. This produces a 1,280-dimensional vector per protein.

The second model is ProtT5 (Rostlab/prot_t5_xl_uniref50) [16]. It uses the T5 encoder-decoder transformer architecture and

was trained specifically on protein sequence data. ProtT5 requires a space-separated input format where each amino acid is treated as a separate token (e.g., "MKTLL" becomes "M K T L L"). The same mean pooling strategy is applied to produce a 1,024-dimensional vector per protein.

A third embedding dataset was created by concatenating the ESM-2 and ProtT5 vectors for each protein, resulting in a 2,304-dimensional representation. All sequences were truncated to a maximum of 1,024 tokens before embedding, which covers 90.8% of sequences without any truncation.

Embedding generation was performed on the High Point University HPC cluster using CPU compute nodes with 128 cores and 512 GB of RAM. The HPC compute nodes have no internet access, so the pretrained models were downloaded separately and stored locally. A Singularity container was built with Python 3.12, PyTorch (CPU), and Hugging Face Transformers to provide a reproducible runtime environment. ESM-2 embeddings took approximately 6 to 8 hours to generate, while ProtT5 required approximately 58 hours.

4.2 XGBoost Classifier

XGBoost is a gradient-boosted decision tree algorithm that builds an ensemble of weak learners (decision trees) sequentially. Each new tree corrects the errors of the previous ones [14]. In this project, one independent XGBClassifier is trained for each of the 1,585 GO terms. Each model performs binary classification: given a protein embedding vector as input, it predicts whether that protein should receive a specific GO term annotation.

The key hyperparameters are: `n_estimators=50` (number of trees), `max_depth=4` (maximum tree depth), `learning_rate =0.3`, and `early_stopping_rounds=5`. Class imbalance is addressed by setting `scale_pos_weight` dynamically for each GO term to the ratio of negative examples to positive examples. This causes the model to assign higher importance to the minority positive class during training. The `tree_method` is set to "hist" for efficient histogram-based splitting, and `n_jobs=-1` to parallelize across all available CPU cores.

Training 1,585 independent models exceeded Google Colab's five-hour session limit. To address this, a second Singularity container was built with XGBoost, scikit-learn, and pandas installed. All three embedding datasets were submitted as parallel Slurm jobs on separate HPC nodes, each completing in approximately 2.5 to 4 hours.

A key limitation of this approach is that each XGBoost model trains independently. It cannot learn patterns shared between related GO terms. For example, if two GO terms frequently co-occur or share a parent in the ontology hierarchy, the XGBoost models for those terms have no way to share this information.

4.3 Multilayer Perceptron (MLP)

The MLP is a two-layer fully connected feedforward neural network built in PyTorch that predicts all 1,585 GO terms simultaneously in a single forward pass. The architecture is as follows:

1. **Input layer**: Takes the embedding vector (1,280 dimensions for ESM-2, 1,024 for ProtT5, or 2,304 for concatenated).
2. **Hidden layer 1**: 1,024 neurons, followed by Batch Normalization, ReLU activation, and 30% Dropout.
3. **Hidden layer 2**: 512 neurons, followed by Batch Normalization, ReLU activation, and 30% Dropout.
4. **Output layer**: 1,585 neurons (one per GO term), with Sigmoid activation applied during inference to produce probabilities between 0 and 1.

Batch Normalization stabilizes training by normalizing activations between layers. Dropout randomly zeros 30% of neurons during each training step, which acts as regularization and prevents the model from memorizing the training data. The output layer bias is initialized to -2.0, so the model starts by predicting near-zero probabilities for all terms. This reflects the extreme label sparsity in the dataset and prevents unstable early training.

The loss function is Focal Loss [17], defined as:

$$FL(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t)$$

where p_t is the predicted probability for the correct class, $\alpha = 0.25$, and $\gamma = 2.0$. Focal Loss addresses class imbalance by downweighting easy, correctly classified examples and focusing training on harder examples.A model could achieve very low loss by simply predicting zero for every term.

The optimizer is AdamW [18] with a learning rate of 0.001 and weight decay of 0.0001. A ReduceLROnPlateau scheduler halves the learning rate when the validation loss stops improving for three consecutive epochs. Training stops early if no improvement is observed for seven consecutive epochs. The batch size is 512.

Unlike XGBoost, the MLP's shared hidden layers allow it to learn general protein features that are useful across multiple GO terms simultaneously. When the network learns that a particular embedding pattern indicates a membrane protein, that knowledge can inform predictions for multiple Cellular Component terms at once.

The MLP was trained in Google Colab using a T4 GPU.

4.4 Ensemble Methods

Two ensemble strategies were evaluated. The first ensemble averaged the MLP and XGBoost predictions for each embedding dataset:

$$P_{\text{ensemble}}(t \mid x) = \frac{P_{\text{MLP}}(t \mid x) + P_{\text{XGBoost}}(t \mid x)}{2}$$

where $P(t \mid x)$ is the predicted probability that protein x has GO term t. This was computed for each of the three

embedding datasets separately, producing three ensemble prediction sets (Experiments 7 through 9 in Table 2). The motivation was that different model types may make different kinds of errors, and averaging could cancel out those errors. However, this assumes both models are comparably strong. If one model is significantly weaker, averaging will dilute the stronger model's predictions.

After evaluating the first nine experiments, it was clear that XGBoost consistently underperformed the MLP by 8 to 10 points across all embeddings. Including XGBoost in the ensemble degraded the MLP's predictions rather than improving them. This motivated a second ensemble strategy: a cross-embedding MLP stack that combines the three MLP models (one per embedding dataset) without any XGBoost involvement:

$$P_{\text{final}}(t \mid x) = \frac{P_{\text{ESM2}} + P_{\text{ProtT5}} + P_{\text{Concat}}}{3}$$

The rationale is that ESM-2 and ProtT5 are independently trained protein language models with different architectures. Even though each MLP has the same architecture, the three models learn from different numerical representations of the same proteins. Because of this, they make largely uncorrelated errors. Averaging their predictions cancels noise that any individual model retains on its own.

4.5 Evaluation Metric

The primary evaluation metric is protein-centric Fmax, the official scoring method used in CAFA challenges [8, 12]. For a given threshold t, each protein's predicted probabilities are binarized: any GO term with probability $\geq t$ is predicted as positive. Precision and recall are then computed for each protein individually and averaged across all proteins:

$$\text{precision}(t) = \frac{1}{N_p(t)} \sum_{i=1}^{N_p(t)} \frac{TP_i(t)}{P_i(t)}, \quad \text{recall}(t) = \frac{1}{N} \sum_{i=1}^{N} \frac{TP_i(t)}{T_i}$$

where $TP_i(t)$ is the number of true positives for protein i at threshold t, $P_i(t)$ is the number of predictions, T_i is the number of true labels, $N_p(t)$ is the number of proteins with at least one prediction, and N is the total number of proteins with at least one true annotation. The F1 score is computed from the averaged precision and recall at each threshold. Fmax is the maximum F1 across 50 thresholds from 0.01 to 0.99.

This process is performed separately for Biological Process, Molecular Function, and Cellular Component. The final CAFA score is the mean of the three Fmax values:

$$\text{CAFA Score} = \frac{\text{Fmax}_{BP} + \text{Fmax}_{MF} + \text{Fmax}_{CC}}{3}$$

All models were evaluated on the same 20% held-out validation set (15,292 proteins), with the remaining 80% (61,166 proteins) used for training. The split was performed with a fixed random seed to ensure identical train and validation sets across all experiments.

5 Experiments, Results, and Discussion

Ten experiments were conducted to systematically compare embedding representations, classification models, and ensemble strategies. The first nine experiments tested three embedding types (ESM-2, ProtT5, and concatenated) against three model configurations (MLP, XGBoost, and an MLP+XGBoost average ensemble). After analyzing these results, a tenth experiment was added: a cross-embedding stack that averages the three MLP predictions together. All experiments were evaluated on the same held-out validation set using protein-centric Fmax, computed separately per ontology.

Table 2. Fmax scores for all ten experiments. Bold values indicate the highest score within each model group and in each column. The MLP Stack achieved the highest score across all columns.

Model	Embedding	BP	MF	CC	CAFA
MLP	ESM-2	0.2754	0.6587	0.5154	0.4832
MLP	ProtT5	**0.2953**	0.6561	**0.5307**	**0.4940**
MLP	Concat	0.2852	**0.6592**	0.5236	0.4893
XGBoost	ESM-2	0.2375	0.4841	0.4366	0.3861
XGBoost	ProtT5	**0.2510**	0.4992	0.4551	0.4018
XGBoost	Concat	0.2505	**0.5065**	**0.4601**	**0.4057**
Ensemble	ESM-2	0.2531	0.5549	0.4751	0.4277
Ensemble	ProtT5	**0.2682**	0.5643	**0.4923**	0.4416
Ensemble	Concat	0.2647	**0.5693**	0.4922	**0.4421**
MLP Stack	All	**0.3101**	**0.6725**	**0.5363**	**0.5063**

The MLP outperformed XGBoost across every embedding type and ontology. The best individual model was the MLP with ProtT5 embeddings, achieving a CAFA score of 0.4940. The best XGBoost model scored only 0.4057. This gap exists because XGBoost trains a separate independent model for each of the 1,585 GO terms, meaning it cannot learn patterns shared between related terms. The MLP uses shared hidden layers that learn general protein features useful across many GO terms at once.

The MLP+XGBoost ensemble failed to improve over the MLP alone. Across all three embeddings, the ensemble scored between 0.4277 and 0.4421. This is worse than every individual MLP model. When two models of very different quality are averaged, the weaker model drags the stronger model's predictions down rather than complementing them. Effective ensembling requires models of comparable strength that make different types of errors.

This finding motivated Experiment 10, the MLP cross-embedding stack. Rather than combining two different model types on the same embeddings, this approach combines three MLP models that were each trained on different embedding representations. ESM-2 and ProtT5 are independently

trained protein language models with different architectures. Because of this, the three MLP models learn from different numerical views of the same proteins and make largely uncorrelated errors. Averaging their predictions cancels noise that any individual model retains on its own. The MLP stack achieved the highest score of any experiment at 0.5063. This is an improvement of 0.012 over the best single MLP (ProtT5, 0.4940) and exceeds the project target of 0.47 by a substantial margin.

ProtT5 was the strongest individual embedding type. It outperformed ESM-2 on both Biological Process and Cellular Component while performing comparably on Molecular Function. Concatenating both embeddings did not improve over ProtT5 alone for any individual model. This suggests that the two models capture overlapping information and the extra dimensions introduce noise without adding useful signal. However, when combined in the MLP stack, the concatenated embedding still contributed positively. The stack with all three embeddings outperformed a stack using only ESM-2 and ProtT5.

Across all experiments, Biological Process was the hardest ontology to predict (scores between 0.24 and 0.31). Molecular Function was the easiest (0.48 to 0.67). This is consistent with prior work, as molecular function is closely tied to the amino acid sequence. Biological process depends on higher-level cellular interactions that are harder to learn from sequence alone. Three individual MLP models and the MLP stack all exceeded the project target of 0.472. No XGBoost or MLP+XGBoost ensemble model reached it.

6 Conclusions and Future Work

This project developed a machine learning pipeline for predicting Gene Ontology functional annotations from protein amino acid sequences. The pipeline combined pretrained protein language models (ESM-2 and ProtT5) for feature extraction with two classification approaches (MLP and XGBoost). Ten experiments were conducted, producing a systematic comparison of embedding types, classifiers, and ensemble strategies, all evaluated using the CAFA protein-centric Fmax metric.

The best-performing configuration was the MLP cross-embedding stack, which averaged the predictions of three MLP models trained on ESM-2, ProtT5, and concatenated embeddings. It achieved a CAFA score of 0.5063 on the held-out validation set, exceeding the project's target benchmark of 0.472 by a substantial margin. The MLP consistently outperformed XGBoost across all embedding types, demonstrating that a shared multi-label architecture capable of learning common patterns across related GO terms is more effective than training independent classifiers for each term. The MLP+XGBoost ensemble failed to improve over any individual MLP model because XGBoost was 8 to 10 CAFA points weaker. Averaging models of very different quality drags the stronger model down rather than complementing it. The MLP stack succeeded where the MLP+XGBoost ensemble failed because all three MLP models were of comparable strength but learned from different numerical representations. This caused them to make different mistakes, and averaging cancelled out those errors.

Molecular Function was the strongest ontology across all models, while Biological Process was consistently the weakest. This reflects the indirect relationship between sequence features and higher-level biological processes. Concatenating ESM-2 and ProtT5 embeddings did not improve performance over ProtT5 alone for any individual model, suggesting the two representations capture overlapping information. However, the concatenated embedding still contributed positively when included in the MLP stack, as the stack with all three embeddings outperformed a stack using only ESM-2 and ProtT5.

This project was inspired by the Kaggle competition CAFA-6 Protein Function Prediction, which served as the source of the dataset used in this work. The competition's entry deadline closed in January 2026, just as this project began. The CAFA evaluation process includes an additional test set of 224,309 unlabeled protein sequences that participants' models must generate predictions for. After the prediction deadline, the organizers wait several months for new experimentally verified annotations to accumulate in public databases from laboratories worldwide. These newly confirmed annotations are then used as ground truth to evaluate how well each model predicted the functions of previously unannotated proteins. Although the best-performing models from this project were run on the test data, there is no way to verify the accuracy of those predictions without access to the official evaluation. Future work would include submitting predictions to a future CAFA challenge to obtain an independent score and benchmark the models against other approaches in the field.

There are also several ways to improve the models themselves. The first would be expanding the number of GO terms used for training. This project filtered the label space down to 1,585 terms to make training feasible, but other teams have used much larger sets. For example, the ProtBoost team used 4,500 GO terms in their CAFA-5 submission. A larger label space would give the model more fine-grained predictions and could improve coverage on rare functions. The second improvement would be replacing XGBoost with a different gradient boosting model. Standard XGBoost struggles on this task because it trains one independent classifier for each GO term and cannot learn shared patterns between related terms. A multi-output gradient boosting library like Py-Boost would solve this problem by training a single model that predicts all GO terms at once, similar to how the MLP does. A stronger boosting model could also make the MLP+XGBoost ensemble viable, since the ensemble failed only because XGBoost was too weak relative to the MLP.

The third improvement would be using a larger protein language model. This project used the 650 million parameter version of ESM-2, but a 3 billion parameter version also exists. Larger models usually produce richer embeddings and capture more complex biological patterns, which could push performance higher. The fourth improvement would be adding protein structure information from AlphaFold2 to the input features. Papers like DPFunc have shown that combining sequence embeddings with predicted 3D structure data improves function prediction results.

References

[1] Maxat Kulmanov and Robert Hoehndorf. 2021. Protein Function Prediction with Gene Ontology: From Traditional to Deep Learning Models. *PeerJ* 9, Article 12019. https://doi.org/10.7717/peerj.12019

[2] Mahakaran Sandhu et al. 2025. Investigating the Determinants of Performance in Machine Learning for Protein Fitness Prediction. *Protein Science* 34, 8, Article e70235. https://doi.org/10.1002/pro.70235

[3] Wenkang Wang et al. 2025. DPFunc: Accurately Predicting Protein Function via Deep Learning with Domain-Guided Structure Information. *Nature Communications* 16, Article 70. https://doi.org/10.1038/s41467-024-54816-8

[4] Lu Meng and Xiaoran Wang. 2024. TAWFN: A Deep Learning Framework for Protein Function Prediction. *Bioinformatics* 40, 10, Article btae571. https://doi.org/10.1093/bioinformatics/btae571

[5] Amelia Villegas-Morcillo et al. 2021. Neural Networks to Learn Protein Sequence-Function Relationships from Deep Mutational Scanning Data. *Proceedings of the National Academy of Sciences* 118, 48, Article e2104878118. https://doi.org/10.1073/pnas.2104878118

[6] Tian-Ci Yan et al. 2022. PPVED: A Machine Learning Tool for Predicting the Effect of Single Amino Acid Substitution on Protein Function in Plants. *Plant Biotechnology Journal* 20, 8, 1620–1632. https://doi.org/10.1111/pbi.13823

[7] Frimpong Boadu et al. 2025. Deep Learning Methods for Protein Function Prediction. *Proteomics* 25, 2, Article e2300471. https://doi.org/10.1002/pmic.202300471

[8] Predrag Radivojac. 2013. A (Not So) Quick Introduction to Protein Function Prediction.

[9] Iddo Friedberg and Predrag Radivojac. 2017. Community-Wide Evaluation of Computational Function Prediction. *Methods in Molecular Biology* 1446, 133–146.

[10] Predrag Radivojac et al. 2013. A Large-Scale Evaluation of Computational Protein Function Prediction. *Nature Methods* 10, 3, 221–227.

[11] Yuxiang Jiang et al. 2016. An Expanded Evaluation of Protein Function Prediction Methods Shows an Improvement in Accuracy. *Genome Biology* 17, 1, Article 184.

[12] Naihui Zhou et al. 2019. The CAFA Challenge Reports Improved Protein Function Prediction and New Functional Annotations for Hundreds of Genes Through Experimental Screens. *Genome Biology* 20, 1, Article 244.

[13] Leonid Iosipoi and Anton Vakhrushev. 2022. SketchBoost: Fast Gradient Boosted Decision Tree for Multioutput Problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35.

[14] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 785–794. https://doi.org/10.1145/2939672.2939785

[15] Zeming Lin et al. 2023. Evolutionary-Scale Prediction of Atomic-Level Protein Structure with a Language Model. *Science* 379, 6637, 1123–1130. https://doi.org/10.1126/science.ade2574

[16] Ahmed Elnaggar et al. 2022. ProtTrans: Toward Understanding the Language of Life Through Self-Supervised Learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 10, 7112–7127. https://doi.org/10.1109/TPAMI.2021.3095381

[17] Tsung-Yi Lin et al. 2017. Focal Loss for Dense Object Detection. In *Proceedings of the IEEE International Conference on Computer Vision*. 2980–2988. https://doi.org/10.1109/ICCV.2017.324

[18] Ilya Loshchilov and Frank Hutter. 2019. Decoupled Weight Decay Regularization. In *International Conference on Learning Representations (ICLR)*.

[19] Adam Paszke et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems*, Vol. 32. 8024–8035.

[20] Thomas Wolf et al. 2020. Transformers: State-of-the-Art Natural Language Processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. 38–45. https://doi.org/10.18653/v1/2020.emnlp-demos.6

[21] Quinn Ramsay. 2026. Protein Function Prediction Model. GitHub repository. https://github.com/quinnpramsay/Protein-Function-Prediction-Model